

PROJECT BRIEF · MAY 2026

SATM — Smart Academic Task Manager

An AI-powered web application that helps students organize, prioritize, and schedule academic tasks using machine learning.

Python · Flask

scikit-learn · NLTK

9 ML Models

Deployed on Render

WHAT IS SATM?

Students often struggle to figure out which task to do first, how long it will take, and how important it is relative to everything else on their plate. SATM solves this by letting a student describe any academic task in plain language — the way they'd tell a friend — and the AI does the rest.

The app classifies the task, estimates how long it will take, scores its urgency, and builds a prioritized daily schedule automatically. Everything runs in the browser; no installation needed.

HOW A USER USES IT — STEP BY STEP

1

Register or log in

Create an account with an email and a strong password. The app validates the password live (length, uppercase, number, special character). Sessions persist across visits.

2

Type your task in plain English

Write it exactly how you'd say it: *"finish bio hw ASAP"*, *"study for databases exam in 3 days"*, *"submit ML project by next week"*. No dropdowns, no forms.

3

Deadline is auto-detected

The app reads your sentence and extracts the deadline automatically — "ASAP" → 1 day, "tomorrow" → 1 day, "in 3 days" → 3, "next week" → 7. You can always override it with the number field.

4

Press Analyze — the AI pipeline runs

The text goes through NLP preprocessing (cleaning, tokenization, stopwords removal, lemmatization) and then three separate ML prediction stages, each using an ensemble of 3 models. See the AI section below for exactly what happens here.

5

Results appear — with model agreement indicators

The app shows: **Category** (Homework / Exam / Quiz / Project), **Importance** (High / Medium / Low), **Estimated time** (hours). Next to each field is a badge showing how many of the 3 models agreed — "3/3 agree" in green, "2/3 agree" in orange. This tells the user how confident the prediction is.

6

Adjust if needed, then add to task list

All three detected fields are editable dropdowns — the user can correct the AI if it got something wrong. A priority score (0–100) and recommendation are shown before they click "Add to task list."

7

View prioritized list and auto-generated schedule

All saved tasks are sorted by urgency × importance. The Schedule tab generates a full daily study plan with 45-minute sessions and automatic breaks, fitted to the tasks' estimated times.

THE AI PIPELINE — MOST IMPORTANT SECTION

Every task runs through three prediction stages in sequence. Each stage is an independent ensemble of three machine learning models. This is the core of the project.

Step 1 — Text Preprocessing (NLP)

Before any model sees the text, it goes through a pipeline built with **NLTK**:

- **Cleaning** — lowercasing, removing punctuation and emojis
- **Abbreviation expansion** — "hw" → "homework", "bio" → "biology", "ppt" → "presentation"
- **Tokenization** — splitting the sentence into individual words
- **Stopword removal** — dropping words like "the", "is", "a" that carry no meaning
- **Lemmatization** — reducing words to their root form ("studying" → "study", "exams" → "exam")

The cleaned text is then converted into a numerical vector using **TF-IDF** (Term Frequency–Inverse Document Frequency), which scores how important each word is relative to the whole training dataset.

Step 2 — Three-Stage Ensemble Prediction**STAGE 1****Category**

Input: TF-IDF vector

- Random Forest
- LinearSVC (SVM)
- Gradient Boosting

Majority vote

→ Homework / Exam / Quiz / Project

STAGE 2**Importance**

Input: TF-IDF + deadline

- Random Forest
- LinearSVC (SVM)
- Gradient Boosting

Majority vote

→ High / Medium / Low

STAGE 3**Time Estimate**

Input: TF-IDF + category

- Random Forest
- LinearSVR (SVM)
- Gradient Boosting

Average of three

→ X.X hours (0.5–8)

Why an ensemble? A single model can be confidently wrong. By running three independent models and combining their answers — majority vote for classification, average for regression — we reduce that risk. If all three agree, we show "3/3 agree" in green. If only two agree, we show "2/3 agree" in orange, signalling the user to double-check.

Why these three models?

- **Random Forest** — an ensemble of decision trees itself; handles sparse TF-IDF vectors well, resistant to overfitting
- **SVM (Support Vector Machine)** — finds the widest margin between classes; historically the strongest performer on text classification tasks
- **Gradient Boosting** — builds trees sequentially, each correcting the previous one's errors; captures complex patterns the other two might miss

Training Data & Accuracy

All models were trained on a labelled dataset of **1,000 academic tasks**. The dataset was split 80% training / 20% test, stratified by category so every category is represented equally in both splits.

Prediction	Random Forest	SVM	Gradient Boosting	Note
Category	99.5%	99.5%	99.0%	Classification — 4 classes
Importance	100%	99.5%	100%	Classification — 3 classes
Time estimate	MAE ≈ 57 min	MAE ≈ 58 min	MAE ≈ 57 min	Regression — continuous output

Time estimation accuracy is lower by design — this is a *regression* problem (predicting a continuous number), not classification. The same task genuinely takes different students different amounts of time, so there is real noise in the labels. The three models agree closely with each other (within ~1 minute on average), and their average prediction is better than any single model alone. $R^2 \approx 0.50$, meaning the ensemble captures about half the real variation in task duration — which is reasonable given the data size and inherent unpredictability.

TECH STACK — QUICK OVERVIEW



Frontend

Single HTML file — vanilla JavaScript, no frameworks. Dark UI with live validation and dynamic task list. Communicates with the backend via fetch (REST API).



Backend

Python + Flask. Handles authentication, runs the ML pipeline, and serves the API. Runs on gunicorn in production. Deployed on Render.



Database

SQLite via Flask-SQLAlchemy. Two tables: Users (id, email, hashed password) and Tasks (linked to user, stores all predictions and status).

**Authentication**

Session-based auth. Passwords hashed with Werkzeug (bcrypt). Registration enforces email format + strong password rules. Sessions persist across browser visits.

**ML / NLP**

scikit-learn for all models and TF-IDF. NLTK for tokenization, lemmatization, and stopwords. scipy.sparse for memory-efficient feature matrices.

**Deployment**

Deployed on Render. Secret key and DB URI managed via environment variables.